

Corporate Capstone Project 2026

ARIA

Agentic Real-Estate Intelligence Advisor

A multi-agent machine-learning and generative-AI system for short-term-rental pricing, risk, demand, and compliance intelligence across Paris and Athens.

FINAL CAPSTONE REPORT

Prepared by: Álvaro Jiménez · Álvaro Pérez · Andrea Alarcón · Bassem Halawani · Ghezlan Almatar · Luka Tcheishvili

Programme: MBDS

Mentor: Angela He Chen

Company: KPMG Spain

Date: June 2026

Repository: github.com/lukatcheishvili/capstone_project_kpmg

Live demo: <http://capstone-project-kpmg-git-main-lukatcheishvilis-projects.vercel.app>

EXECUTIVE SUMMARY

ARIA turns 135,051 public Airbnb listings across Paris and Athens into explainable, model-grounded recommendations for three decision-makers: institutional investors, hosts, and compliance advisors who currently have no systematic way to know who is underpriced, who is at risk, and who is operating outside the rules. Four validated models — XGBoost pricing, LightGBM host-risk, Prophet demand, and a retrieval-augmented compliance agent — are wired behind a nine-node LangGraph orchestrator with a human-in-the-loop checkpoint and exposed through a live, grounded conversational interface backed by Vertex AI. The architecture's defining discipline is strict: the structured models compute every number, and the language model only narrates and explains those verified outputs in business language — it is never the source of a number, which is ARIA's core defence against hallucination in a system built to inform real pricing and investment decisions.

Every model met or exceeded its pre-declared validation target on a locked holdout, with no exceptions: the Athens XGBoost pricing model reaches $R^2 = 0.676$ (target >0.65), the corrected LightGBM risk classifier reaches $AUC-ROC = 0.8288$ (target >0.72), and the LangGraph orchestrator achieved 100% persona-routing accuracy. The consolidated business signal is approximately €7.7M of identified annual opportunity and exposure in Athens alone, anchored by a single decision-ready output: 865 listings that are simultaneously underpriced and high-risk.

ARIA is delivered as a working, live system rather than a static analysis: the same models, grounding logic, and human-in-the-loop safeguard demonstrated in this report are running behind the deployed conversational interface today, available for an investor, host, or regulator to query directly rather than read about secondhand.

KEY RESULTS AT A GLANCE

135,051	865	€7.7M	100%
LISTINGS ANALYSED <i>Paris + Athens · 96 engineered features</i>	PRIORITY ACTION LIST <i>Underpriced AND high-risk overlap</i>	IDENTIFIED OPPORTUNITY <i>Annual value: pricing, risk, demand, compliance</i>	VALIDATION TARGETS MET <i>Every model, locked holdout, no exceptions</i>

Ethics Statement

ARIA is a decision-support prototype, not an autonomous decision-maker. Every output — a fair-price estimate, a risk probability, a demand forecast, or a compliance flag — is a signal for a human host, investor, or advisor to weigh against their own judgement, never an instruction executed automatically. This is enforced in the pipeline: the LangGraph orchestrator includes an explicit human-in-the-loop interrupt before any recommendation is finalised, and the interface labels every numeric claim with its source and limitations. The project uses exclusively public, anonymised listing-level data from Inside Airbnb, Maven Analytics, and the Gyódi and Nawaro (2021) dataset; no personally identifiable information is collected or inferred beyond what these sources already disclose, and no individual guest is profiled.

We make two governance disclosures transparently. First, the Athens host-risk label (`at_risk_host`) is an engineered six-dimension proxy, not an observed ground truth, since no public source records host churn — and we report performance against it honestly, including its limits. Second, an early version of the risk model produced an implausible AUC-ROC of 0.9995; we used SHAP to diagnose target leakage, removed the leaking features, and reported the corrected, honest 0.8288 throughout. We treat this correction as a demonstration of Trusted AI discipline, not a result to hide.

Remaining limits are disclosed rather than smoothed over. The compliance module flags listings that appear unlicensed under the Athens AMA framework as analyst-triage support only — never a final legal determination or accusation against any named host. Paris pricing partly relies on 2021 and estimated-price data, so those rows are excluded from training and Paris conclusions are flagged as more provisional than Athens. VADER sentiment shows a measurable positive bias on French-language reviews, so cross-city sentiment comparisons are suppressed at the data layer. Finally, the generative-AI layer is constrained by design to narrate verified model outputs — it is instructed never to invent counts, scores, or values, and every response closes with a verifiable source line, keeping a structured-models-compute, language-model-explains separation as the core safeguard against hallucination.

AI Usage Disclaimer

In the interest of academic transparency, we disclose that generative-AI assistants (large language models) were used as productivity tools during the development of this capstone. Their use was confined to supporting tasks: scaffolding and debugging code, drafting and refactoring docstrings and comments, accelerating routine data-wrangling boilerplate, and assisting with the drafting, copy-editing, and formatting of this written report. We treated every such suggestion as a draft to be checked, not an authority to be trusted.

All substantive intellectual contributions of this project are our own. Every problem definition, data-governance decision, modelling choice, hyperparameter and validation strategy, leakage audit, metric, monetary estimate, and conclusion in this report was designed, executed, reviewed, and verified by us as a team; no figure or claim was accepted because a model produced it. Where a tool contributed to code or prose, we read, tested, and took full responsibility for the result.

This disclaimer concerns our own use of AI *while building the project*. It should not be confused with ARIA's embedded generative-AI layer (Vertex AI, by using models of gemini and claude), which is a component of the delivered system and is governed by the grounding and human-in-the-loop safeguards described in the Ethics Statement above and in Sections 4.5 and 5.6.

Table of Contents

1 • Objective	3
1.1 The Business Problem · 1.2 Aim of the Work · 1.3 Why Two Cities · 1.4 The Advanced Track	
2 • Data Sources	5
2.1 Inventory · 2.2 City Treatment · 2.3 Preparation Pipeline · 2.4 Additional Data · 2.5 Output Contracts	
3 • Methodology	7
3.1 Approach · 3.2 Tooling · 3.3 Phase-by-Phase · 3.4 Validation Discipline	
4 • Potential Solutions	10
4.2 Pricing Family · 4.3 Risk Target · 4.4 Risk Algorithm · 4.5 Role of the LLM · 4.6 Summary	
5 • Development	13
5.2 Pricing Code · 5.3 Tuning · 5.4 Compliance · 5.5 Orchestration · 5.6 Grounding · 5.7 Practices	
6 • Conclusions	15
6.1 Result Summary · 6.2 Pricing · 6.3 Risk · 6.4 Forecast/Compliance · 6.5 Impact · 6.6 Practicality	
7 • Governance, Limitations & Next Steps	18
7.1 MLOps Maturity · 7.2 Limitations · 7.3 Next Steps · 7.4 References	

1 Objective

1.1 The Business Problem

Short-term rental (STR) platforms such as Airbnb generate enormous volumes of public listing data, yet very little of it is converted into decision-ready intelligence for the three groups who most need it: hosts who do not know whether they are leaving revenue on the table, investors who cannot easily compare neighbourhood-level entry opportunities, and city regulators or advisory firms who lack a systematic way to identify non-compliant supply. This project explores whether a combination of classical machine learning, time-series forecasting, retrieval-augmented generation, and large-language-model orchestration could close that gap, turning raw listing data into an explainable, multi-persona advisory product that mirrors how a professional advisory engagement would actually be delivered to a client.

Our project, named ARIA (Agentic Real-Estate Intelligence Advisor), was scoped around three personas whose questions cannot be answered by a single model or a single dashboard:

- **Institutional investor / PE fund:** "Which neighbourhood should I enter, what yield can I expect, and what is the downside risk?"
- **Host / property manager:** "Am I priced correctly? Is my listing's performance declining? What should I change?"
- **Regulator / compliance advisor:** "Which listings in my city appear to operate without a licence, and where is the exposure concentrated?"

1.2 Aim of the Work

The aim of the project is to design, build, and validate an end-to-end pipeline from raw public data acquisition through to a deployed conversational interface that produces explainable, model-grounded answers to the three questions above for two structurally different European short-term-rental markets: Paris and Athens. Concretely, the work set out to:

- Construct a single, unified, feature-engineered dataset spanning two cities and three independent public data sources, with documented data-quality decisions and routing flags that protect each downstream model from training on unreliable rows.
- Train and validate a fair-price prediction model (XGBoost) per city, and use it to quantify a concrete, monetisable underpricing opportunity.
- Train and validate a host/listing risk classifier (LightGBM) that flags vulnerable hosts using only information genuinely available before the fact, with an explicit, documented audit for data leakage.
- Forecast neighbourhood-level short-term demand twelve months forward (Prophet) to give the pricing and risk outputs a forward-looking context.
- Build a retrieval-augmented compliance agent (ChromaDB plus a regulatory corpus) that connects individual listings to the specific municipal regulation they may be violating.
- Orchestrate all four specialist models behind a single conversational agent (LangGraph) with a human-in-the-loop checkpoint, and productionise a compatible web experience (Vercel and Vertex AI by using Gemini or Claude models) that a non-technical stakeholder can use unaided.
- Express every model output in business language and euros, not only in statistical metrics, so that the system's business value is self-evident.

1.3 Why Two Cities, and Why These Two

Paris and Athens were chosen deliberately rather than arbitrarily. They give the project two markets that are structurally different enough to require independent modelling decisions, which is itself an important methodological finding while both being large, mature Airbnb markets with multiple available public data sources. Paris contributes scale (120,809 listings, spanning a 2021 source with observed prices and a 2025 source with current behavioural signals) and a flatter, more dispersed tourist-demand geography. Athens contributes currency and completeness (14,242 listings, all dated September 2025, with observed prices, licence status, and calendar-derived behavioural signals in a single internally consistent snapshot) and a much sharper centre-to-periphery price gradient. The exploratory analysis (Section 2) shows that the distance-to-centre effect on price is roughly 2.8 times stronger in Athens than in Paris, a quantified justification, not an assumption, for why the project trains separate per-city models rather than one pooled model.

1.4 Scope and the Advanced Track

The project pursued the advanced, agentic multimodal track rather than stopping at a conventional pricing and forecasting pipeline. In addition to the two required predictive models, we built a retrieval-augmented regulatory compliance agent, a nine-node LangGraph multi-agent orchestrator with checkpointed human-in-the-loop approval, and a deployed conversational front end backed by a server-side large language model call that is itself grounded in, and constrained to, the validated model outputs. This scope was chosen because the business problem is not actually a single prediction task: a real-world advisory deliverable would need to combine pricing, risk, forecasting, and regulatory context into one coherent recommendation, and the agentic architecture is the part of the project that demonstrates whether that combination can be made to work end-to-end, rather than only model by model in isolation.

2 Data Sources

2.1 Source Inventory

No single public dataset contains everything the project needs: observed prices, current availability and demand signals, host behavior, review history, geographic context, and license status. ARIA therefore combines four independent public sources into one master dataset of 135,051 listings and 96 columns, covering Paris (120,809 listings) and Athens (14,242 listings).

Source	City	Rows	Vintage	Contribution
Source	City	Rows	Vintage	Contribution
Maven Analytics	Paris	63,520	2021	Large Paris base with observed prices and rich listing attributes
Inside Airbnb	Paris	57,289	Sept 2025	Current Paris demand, availability, host-behavior signals (price partly estimated)
Inside Airbnb	Athens	14,242	Sept 2025	Current Athens prices, license status, reviews, availability — the single most reliable extract
Gyódi & Nawaro (2021)	Both	enrichment	2021	Zone-level lifestyle, restaurant, attraction, and metro-distance context (Zenodo 4446043)

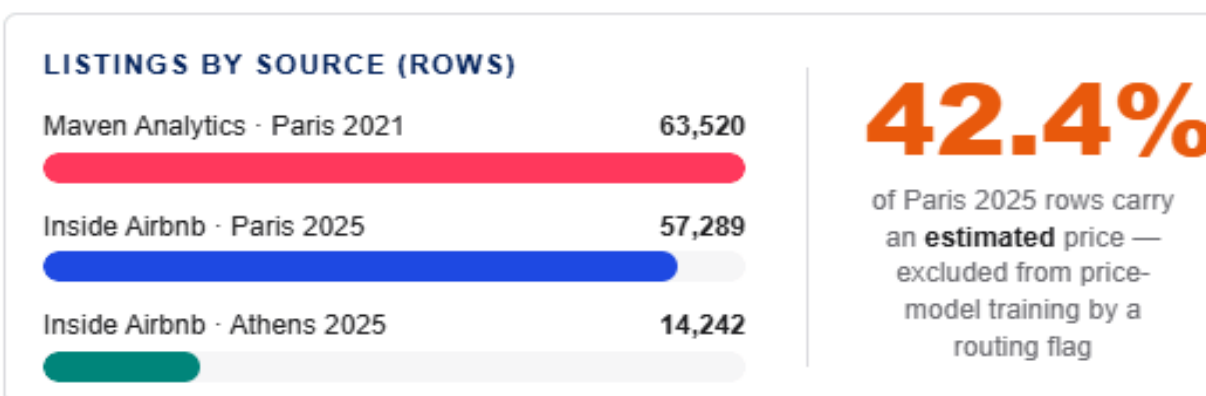


Figure 2.1. Dataset composition by source and the share of Paris rows with estimated rather than observed prices. The estimated-price share is why a routing flag excludes those rows from price-model training (Section 2.3).

2.2 Why Athens and Paris Receive Different Treatment

Athens is the smaller city by row count, but it is the more reliable city for current decision-making: every Athens row originates from a single September 2025 Inside Airbnb snapshot, with observed prices, licence numbers, full calendar-derived availability, and review history in one internally consistent extract. Paris is larger (because it combines a 2021 Maven Analytics extract with a 2025 Inside Airbnb extract) but structurally messier: the 2021 source has observed prices but ageing behavioural signals, while the 2025 source has current behavioural signals but does not publish price for roughly 42% of listings, requiring those prices to be estimated from comparable nearby listings. The project's central data-governance decision is to keep Paris in scope for breadth and cross-market comparison, while treating every Paris pricing conclusion as more provisional than the equivalent Athens conclusion, and to never train a price model on an estimated price.

2.3 Data Preparation and Processing Pipeline

All preparation logic lives in a single, fully executed exploratory-data-analysis notebook (ARIA_EDA_v4_FINAL.ipynb) that documents 41 discrete pipeline steps and writes the unified

master dataset, `aria_mega_dataset_v4_1_final.csv`, to `data/processed/`; every downstream notebook reads from this single file, so there is exactly one place where a data-quality decision can be made incorrectly, and exactly one place to audit it. The three raw sources used different character encodings and different conventions for boolean and price fields (for example, 't'/f strings versus native booleans), resolved before any analysis to prevent silent join corruption.

Short-term rental prices are heavily right-skewed, so the pipeline creates `log_price = log1p(price_eur)` as the modelling target, converting predictions back to euros with `expm1()` before any number reaches a business user. Distance from each listing to its city centre is computed with the Haversine great-circle formula and bucketed into four zones (centre 0–1 km, near 1–3 km, mid 3–5 km, far 5 km+); because this effect is strongly non-linear and city-specific (Section 1.3), the pipeline also engineers `dist_km_sq`, `log_dist_km`, and `inv_dist_km` so downstream tree models can exploit whichever functional form fits each city best.

VADER sentiment scoring was run across 594,482 guest reviews; we discovered and documented that this English-tuned lexicon inflates French-language Paris review scores by an average of +0.178 on a 0–10 scale, so a `sentiment_city_comparable` routing flag suppresses any cross-city sentiment comparison at the data layer rather than leaving it to be misused downstream. Three further engineered fields make the rest of the project possible: `review_score_composite` rebuilds Paris's and Athens's differently structured review fields onto one common 0–10 scale; `at_risk_host` is a transparent proxy label for host vulnerability, since no public source contains a verified churn outcome (Section 4.3 details its construction and leakage correction); and `xgb_price_training_eligible/prophet_training_eligible` are routing flags computed once and referenced by every downstream model, so an estimated-price row, for example, is structurally excluded from price-model training rather than relying on each notebook author to remember to filter it.

2.4 Acquisition of Additional Data

Two enrichment steps extend the raw listings beyond what Inside Airbnb and Maven Analytics publish directly. The Gyódi and Nawaro (2021, Zenodo record 4446043) academic dataset contributes zone-level lifestyle, restaurant-density, attraction-proximity, and metro-distance scores for Paris and Athens, joined at the distance-zone grain rather than the individual-listing grain (these columns therefore have only 3–4 unique values per city, which is expected and documented rather than a data-quality defect). The regulatory corpus used by the compliance agent (Section 2.5) was manually assembled from the Athens AMA short-term-rental licensing framework and the French Loi Le Meur, since no public API or structured dataset of either regulation exists; this corpus is the retrieval index queried by the Phase 5 RAG agent.

2.5 Output Contracts

Each of the four predictive phases writes one or more output files to `data/outputs/`, joined on `listing_id`, which the LangGraph orchestrator, the Vercel backend, and this report all read from the same source of truth. The most important of these are summarised below; the complete contract, including required columns and consuming components, is documented in `data/README.md` and `models/MODEL_CARD.md` in the project repository.

File	Phase	Contents
paris_predictions_v1.csv / athens_predictions_v1.csv	2	Predicted fair price, actual price, and gap for every listing
athens_underpricing_v1.csv	2	2,945 Athens listings flagged where predicted price exceeds actual by more than €15
shap_paris_v1.csv / shap_athens_v1.csv	2	Per-feature SHAP values for every holdout listing
athens_risk_scores_v1.csv	3	risk_probability (0–1), risk_band, and high_risk_flag for all 14,242 Athens listings
prophet_*_forecast_v1.csv	4	12-month occupied-nights forecast per neighbourhood with 90% confidence interval
rag_unlicensed_report_v1.csv / rag_compliance_index_v1.json	5	Compliance triage handoff for the 137 unlicensed Athens listings, with cited regulation
aria_routing_eval.csv / aria_investor_brief.pdf	6	LangGraph persona-routing evaluation evidence and a generated investor-memo artefact

3 Methodology

3.1 Overall Approach

ARIA is built as a layered pipeline rather than a single model, because the underlying business problem is not one prediction task but four related ones: pricing, risk, demand, and compliance, each with a different target variable, a different appropriate model family, and a different consumer. The methodology therefore proceeds phase by phase, with each phase's output file becoming the next phase's documented input contract (Section 2.5), and concludes with an orchestration layer that routes a natural-language question to the correct specialist and a productionisation layer that exposes the result through a conversational interface.

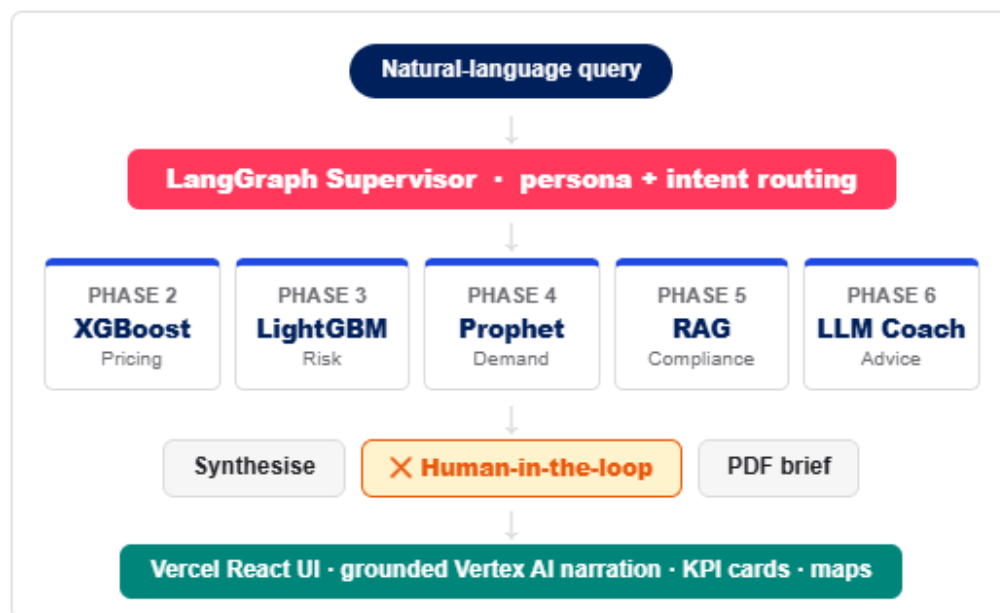


Figure 3.1. ARIA's layered architecture. A single supervisor classifies the query, fans out to the specialist models it needs, synthesises one answer, and pauses at a human-in-the-loop checkpoint before the grounded conversational layer narrates the verified numbers to the end user.

3.2 Tools and Technology Stack

Layer	Tools / Technologies
Data preparation	Python, pandas, NumPy, Jupyter notebooks
Pricing model	XGBoost (XGBRegressor), Optuna, SHAP, scikit-learn
Risk model	LightGBM, Optuna, SHAP, scikit-learn
Demand forecasting	Prophet (Meta), pandas
Compliance retrieval	ChromaDB vector index, manually curated regulatory corpus
Multi-agent orchestration	LangGraph (StateGraph, MemorySaver checkpointing), TypedDict state schema
Generative AI / chat layer	Vertex AI Gemini 2.5 Pro (default), Claude on Vertex, Google Auth service-account flow
Web application	Vite, React, Recharts, Leaflet / OpenStreetMap, Vercel Functions, Vercel hosting
Reporting and export	ReportLab (PDF brief generation), Markdown export
Visualisation (analysis)	matplotlib, seaborn

3.3 Phase-by-Phase Process

3.3.1 Phase 1 — *Exploratory Data Analysis*

Full exploratory analysis across all 135,051 listings: data-quality auditing, the 41-step preparation pipeline described in Section 2.3, and the discovery of the two findings that justify the project's principal modelling decisions — the city-specific distance effect (Section 1.3) and the at-risk host engineering rationale (Section 4.3). Every subsequent modelling choice in this report traces back to a specific, citable finding in this notebook rather than to an unstated assumption.

3.3.2 Phase 2 — *XGBoost Fair-Price Prediction*

Gradient-boosted decision trees (XGBoost) were selected for the pricing task because short-term rental price is governed by non-linear interactions — a two-bedroom entire home in a central neighbourhood behaves nothing like a private room on the periphery, and the size of that gap itself varies by city. This choice was weighed against two explicit alternatives, a regularised linear model and a deep neural network, both considered and rejected on dataset-size and explainability grounds (Section 4.2). Two independent models are trained, one per city, each on a 26-feature pipeline (physical attributes, location and distance transforms, host-quality signals, review and demand proxies, and a `neighbourhood_median_price` local-market benchmark), tuned with Optuna over 100 trials of 5-fold cross-validated search, and evaluated on a locked 20% holdout that is never touched again until final reporting.

3.3.3 Phase 3 — *LightGBM Host Risk Classification*

LightGBM was selected for the Athens risk task because it is an efficient, well-regularised gradient-boosting classifier for structured tabular data with moderate class imbalance (56.8% at-risk versus 43.2% not-at-risk), handled here through LightGBM's native `is_unbalance=True` rather than synthetic oversampling. This algorithm choice was likewise weighed against a logistic-regression baseline and a Random Forest alternative, both considered and rejected in favour of LightGBM's interaction handling and native imbalance support (Section 4.4). The target, `at_risk_host`, is engineered rather than observed (Section 4.3); the feature set is reduced to 11 columns after an explicit leakage audit, and the model is tuned and validated using the same Optuna and stratified cross-validation discipline as Phase 2.

3.3.4 Phase 4 — Prophet Demand Forecasting

Meta Prophet was used to forecast neighbourhood-level occupied-nights twelve months forward for both cities, decomposed into trend and yearly-seasonality components and cross-validated on a rolling-origin basis before the final 12-month projection with 90% confidence intervals was generated. Forecasting was added specifically so that the pricing and risk outputs are not read as static snapshots: a listing that looks merely underpriced today reads very differently to an investor if the forecast direction for its neighbourhood is also rising.

3.3.5 Phase 5 — Retrieval-Augmented Compliance

A ChromaDB vector index was built over a manually assembled regulatory corpus covering the Athens AMA licensing framework and the French Loi Le Meur. Each of the 137 listings identified as unlicensed in Athens is queried against this index, and the agent returns the specific applicable regulation article alongside a compliance risk score, rather than only flagging the listing as a binary problem. This phase exists because pricing and risk analysis alone cannot tell an investor or regulator whether a piece of attractive-looking supply is even legally operable.

3.3.6 Phase 6 — LangGraph Multi-Agent Orchestration

A nine-node LangGraph StateGraph wires all five specialist tools (pricing, risk, forecast, compliance, and an LLM-based listing coach) behind one supervisor node that classifies persona and intent from the user's natural-language query, fans out to whichever specialists the query requires, synthesises a structured response, pauses at an explicit human-in-the-loop interrupt before finalising, and can export a PDF investor brief. State is carried in a 17-field `ARIAState` TypedDict and checkpointed with LangGraph's `MemorySaver`, which is what allows the human-in-the-loop interrupt to actually halt and resume execution rather than merely being a comment in the code.

3.3.7 Phase 7 — Productionised Conversational Interface

The research orchestration layer (Phase 6) is mirrored — in concept and in grounding discipline, though not by directly executing the Python LangGraph runtime — by a live Vite React application deployed on Vercel, with a server-side API route that authenticates to Vertex AI with a service account, fetches the committed model-output CSV and JSON files directly from GitHub, computes a verified analytics pack in JavaScript, and only then calls a large language model (Gemini 2.5 Pro by default, with Claude and alternative Gemini variants selectable) to turn that pack into a written recommendation. This ordering deterministic computation first, language model second, the project's central defence against the system inventing a number it cannot support; the exact grounding mechanism is shown as code in Section 5.6.

3.4 Validation Discipline Applied Across All Phases

- An 80/20 train–holdout split with a fixed random seed (42) for every model, so that results are reproducible across machines and re-runs.
- Stratification by distance zone for the larger, geographically diverse Paris split; simple random splitting for Athens, where the far zone contains only 53 listings and is too small to stratify reliably.
- 100-trial Optuna hyperparameter search using a Tree-structured Parzen Estimator sampler, seeded for reproducibility, for both the pricing and risk models.
- 5-fold (pricing) or stratified 5-fold (risk) cross-validation, with fold-to-fold variance reported explicitly as a stability check rather than only a single point estimate.

- SHAP (Shapley Additive Explanations) computed for every holdout prediction in both the pricing and risk models, providing both global feature-importance ranking and per-listing explanation —the latter being what allows ARIA to tell an individual host why their price recommendation is what it is, not only that it is so.
- An explicit leakage audit before any model result is accepted: every notebook prints a checklist confirming that target-derived columns (the raw price, the log price, and, for the risk model, the exact columns used to construct the `at_risk_host` label) are absent from the feature matrix actually passed to the model.

4 Potential Solutions

4.1 Framing the Design Choice

Several junctures in this project admitted more than one reasonable technical solution. This section presents the alternatives considered at the four points with the greatest influence on the final result, the pricing-model family, the risk-model target and feature set, the risk-model algorithm, and the role of the large language model in the final product, and explains, with reference to the exploratory findings in Section 2 and the validation results in Section 6, why the chosen alternative was selected over the others.

4.2 Choosing the Pricing Model Family

Alternative A — Linear / Regularised Regression (Ridge or Lasso on log_price)

A linear model is the simplest defensible baseline: fast, and trivially interpretable. It was rejected because the EDA found the price-generating process to be strongly non-linear and city-specific — the distance-to-centre effect decays at a different rate in Athens than in Paris, and several features (room type, capacity) interact rather than acting independently. A linear model would require every interaction to be hand-specified in advance, and would still under-fit interactions not anticipated.

Alternative B — A Deep Neural Network on Tabular Features

A network could in principle capture the same non-linearities as a tree ensemble. It was rejected for reasons specific to this project: with roughly 78,000 (Paris) and 11,000 (Athens) training rows, the dataset is small enough that gradient-boosted trees are unlikely to be out-performed; a network is materially harder to explain to the non-technical advisory audience this project targets, undermining the explainability goal behind the SHAP analysis used throughout; and it would require substantially more tuning effort for a benefit the project's own validation results do not suggest would materialise.

Alternative C (Chosen) — XGBoost, One Model Per City

XGBoost handles the non-linear, interaction-heavy, tabular pattern the EDA identified without requiring every interaction to be specified by hand, while remaining compatible with SHAP for global and per-listing explanation. Training one model per city rather than one pooled model with a city indicator is itself evidence-justified: the cross-city SHAP comparison (Figure 6.1) shows Paris pricing is primarily size- and capacity-driven (`person_capacity` and `bedrooms` rank second and third) while Athens pricing is primarily quality- and location-driven (`review_score_composite` and `dist_km` rank second and third) — a single pooled model would blur these genuinely different market mechanics into one averaged, less actionable explanation.

4.3 Choosing the Risk-Model Target and Feature Set

Alternative A — Use an Existing Platform Signal (e.g., Superhost Status) as the Risk Target

Superhost status, review score, or a low-availability flag could each be repurposed directly as a risk indicator without new label engineering. This was rejected because each captures only one narrow dimension of vulnerability and is already observable by the host directly — it would not constitute new intelligence, and none captures the combination of booking-momentum decline, deteriorating satisfaction, and host inexperience that constitutes genuine early-warning risk.

Alternative B — Acquire or Simulate a True Churn Label

The ideal target would be a verified record of host churn. No such label exists in any public Airbnb dataset, and simulating one synthetically (for example, assuming a listing that stops receiving reviews for N months has churned) was rejected, because it would silently encode the our own assumptions about churn as if they were ground truth, with no way to validate that assumption against reality.

Alternative C (Chosen) — A Transparent Six-Dimension Proxy with Explicit Leakage Correction

The project instead engineers `at_risk_host` as an explicit heuristic: a listing is labelled at risk if it trips three or more of six observable warning dimensions (below-median recent review velocity, availability above 200 days per year, negative year-over-year review growth, a guest-satisfaction score below 8.0/10, host tenure under 365 days, and the absence of Superhost status). This is honestly presented as a proxy rather than ground truth, gives the model enough positive examples to learn a meaningful pattern, and outputs a continuous probability rather than only a binary label. The choice was validated rather than assumed: an early version of the classifier returned an implausible AUC-ROC of 0.9995, and SHAP analysis showed the `availability_pressure` feature was leaking the label definition directly into the feature matrix. The leaking features were removed and the model retrained; the honest, corrected AUC-ROC of 0.8288 (Section 6.3) is the number reported throughout this document. We regard this correction — catching and fixing an unrealistically good result rather than presenting it — as itself a demonstration of the Trusted AI discipline this project sets out to model.

4.4 Choosing the Risk-Model Algorithm

Alternative A — Logistic Regression on the Six Warning Dimensions

A logistic regression baseline was considered first for its transparency, speed, and directly interpretable coefficients, which would let an advisor read off exactly how much each warning dimension moves the risk probability. It was rejected because the six engineered dimensions behind `at_risk_host` interact rather than contribute independently — host tenure compounds with declining review velocity in a way a linear decision boundary cannot represent without every interaction being hand-specified in advance, mirroring the same non-linearity argument made against a linear pricing model in Section 4.2.

Alternative B — Random Forest Classifier

A bagged-tree Random Forest classifier was considered as a closer tree-based comparison, since it can represent the same feature interactions as a boosted model without a linear-boundary limitation. It was set aside in favour of LightGBM for two project-specific reasons: LightGBM's leaf-wise growth tends to reach a comparable or better split quality with fewer trees on a dataset of this scale (14,242 Athens rows), and its native `is_unbalance=True` handles the moderate class

imbalance (56.8% at-risk versus 43.2% not-at-risk) directly, without the synthetic oversampling a Random Forest pipeline would otherwise need. Choosing LightGBM for both reasons also keeps the modelling stack consistent: the same Optuna tuning, SHAP explanation, and stratified cross-validation discipline already validated for XGBoost in Phase 2 carries over unchanged to Phase 3.

Alternative C (Chosen) — LightGBM with Native Imbalance Handling

LightGBM was chosen as the well-regularised, interaction-capable, imbalance-aware classifier that the dataset's characteristics call for, while remaining fully compatible with the same SHAP-explainability and Optuna-tuning discipline used everywhere else in the project. The result validates the choice rather than merely assuming it: the corrected classifier reaches $\text{AUC-ROC} = 0.8288$ against a target of >0.72 , with cross-validation fold variance of only ± 0.0088 (Section 6.3), a stable result consistent with the non-linear, interaction-heavy pattern the EDA identified rather than one driven by chance variation across folds.

4.5 Choosing the Role of the Large Language Model in the Final Product

Alternative A — Let the LLM Answer Freely from General Knowledge

The simplest approach sends the user's question directly to an LLM and returns whatever it generates. This was rejected outright for a system whose answers may inform pricing, investment, or compliance decisions: an ungrounded LLM has no access to ARIA's actual model outputs and would either refuse to give specifics or, worse, generate plausible-sounding but fabricated row counts, prices, or risk scores — precisely the hallucination failure mode this project exists to avoid.

Alternative B — Replace the Specialist Models Entirely with LLM Reasoning

A more ambitious alternative would ask the LLM to reason about pricing and risk directly from raw listing features, removing the need for XGBoost or LightGBM. This was rejected because a language model is not a calibrated regressor or classifier: it has no mechanism to guarantee its implicit price estimate is consistent across similar listings, no SHAP-style explanation of which feature drove the number, and no validated accuracy metric reportable in Section 6.

Alternative C (Chosen) — A Grounded LLM Constrained to Narrate Verified Analytics

The chosen architecture keeps the division of labour strict: XGBoost, LightGBM, Prophet, and the RAG retrieval step compute every number, a JavaScript analytics layer assembles those numbers into a verified context pack, and only then is an LLM called — with an explicit system-prompt instruction not to invent row counts, model scores, neighbourhood rankings, or monetary values, and a requirement to close every answer with a verifiable source line. A language model should explain and synthesise in business language, but it should never be the source of a number. The concrete grounding code is shown in Section 5.6, and answer quality is regression-tested against a gold-standard rubric (Section 6.1).

4.6 Summary of Decisions

Decision Point	Chosen Solution	Primary Justification
Pricing model family	XGBoost, separate model per city	Non-linear, city-specific price drivers confirmed by EDA and cross-city SHAP
Risk model target	Engineered, documented <code>at_risk_host</code> proxy	No true churn label exists; proxy is transparent, probabilistic, and leakage-audited

Decision Point	Chosen Solution	Primary Justification
Risk model algorithm	LightGBM, native imbalance handling	Outperforms linear/RF baselines on interaction handling and class imbalance (AUC-ROC 0.8288)
LLM's role in the product	Grounded narrator over verified analytics	Avoids hallucinated prices/scores in a decision-support system

5 Development

5.1 Code Organisation and Reproducibility

Analytical code is organised as self-contained Jupyter notebooks under `eda/`, one per pipeline phase, each writing its outputs to `data/outputs/` or `models/` for the next phase to consume; the conversational front end is a Vite React application with a server-side API route under `Stage 7 - UI Interface/vercel_vite_app/api/`. Every notebook fixes one random seed (`RANDOM_STATE = 42`) across the train/holdout split, all cross-validation folds, and the Optuna sampler, and locates the repository root programmatically rather than via a hard-coded path, so the same notebook runs unmodified on any machine. The excerpts below are taken directly from the executed project code, lightly trimmed for length.

5.2 Phase 2 — XGBoost Pricing: Feature Engineering and Leakage Check

The feature-preparation function below is run once per city. It engineers the distance and interaction features motivated in Section 2.3, computes the neighbourhood market-benchmark feature, and — critically — the notebook prints an explicit leakage assertion immediately afterward, checking that none of the five target-derived columns ended up in the feature list by mistake:

```
def prepare_features(df_city, city_name, features=FEATURES, target=TARGET):
    """Clean feature matrix for a single city training set."""
    df_city = df_city.copy()
    df_city["dist_km_sq"] = df_city["dist_km"] ** 2
    df_city["inv_dist_km"] = 1 / (df_city["dist_km"] + 0.1)
    # Neighbourhood market-context benchmark
    nb = df_city.groupby("neighbourhood")["price_eur"].transform("median")
    df_city["neighbourhood_median_price"] = nb.fillna(df_city["price_eur"].median())
    df_city["room_x_capacity"] = df_city["room_type_encoded"] * df_city["person_capacity"]
    X, y = df_city[features].copy(), df_city[target].copy()
    for col in X.columns[X.isnull().any()]:
        X[col] = X[col].fillna(X[col].median())
    assert X.isnull().sum().sum() == 0
    return X, y

# Leakage check – printed and asserted, not assumed
for col in ["price_eur", "log_price", "price_eur_estimated",
            "xgb_price_training_eligible", "estimated_revenue_1365d"]:
    assert col not in FEATURES, f"LEAKAGE: {col} found in FEATURES"
```

5.3 Hyperparameter Search and Final Model Training

Both the pricing and risk models follow the same pattern: an Optuna objective function performs 5-fold (or stratified 5-fold) cross-validation inside the training set only, the best parameters are then used to fit the final model against the locked holdout, and the result is checked against a pre-declared target benchmark before being accepted:

```
def objective_paris(trial):
    params = {
```

```

        "n_estimators": trial.suggest_int("n_estimators", 100, 2000),
        "max_depth": trial.suggest_int("max_depth", 3, 12),
        "learning_rate": trial.suggest_float("learning_rate", 0.005, 0.3),
        "objective": "reg:squarederror", "random_state": RANDOM_STATE,
    }
    kf, scores = KFold(5, shuffle=True, random_state=RANDOM_STATE), []
    for tr, val in kf.split(X_p_train):
        m = XGBRegressor(**params, early_stopping_rounds=50)
        m.fit(X_p_train.iloc[tr], y_p_train.iloc[tr],
              eval_set=[(X_p_train.iloc[val], y_p_train.iloc[val])], verbose=False)
        scores.append(np.sqrt(mean_squared_error(
            y_p_train.iloc[val], m.predict(X_p_train.iloc[val]))))
    return np.mean(scores)

study = optuna.create_study(direction="minimize",
                           sampler=optuna.samplers.TPESampler(seed=RANDOM_STATE))
study.optimize(objective_paris, n_trials=100)
pred_paris_eur = np.expml(xgb_paris.predict(X_p_val)) # back to euros

```

5.4 Retrieval-Augmented Compliance Agent (Phase 5)

The compliance agent queries the ChromaDB regulatory index, then merges the retrieved regulation with deterministic, zone-based risk rules, so the output is always traceable to a specific citation rather than to free-form generation:

```

RISK_RULES = {
    "centre_0_1km": ("HIGH", "Cannot regularise (AMA freeze Dec 2024)."),
    "near_1_3km": ("MEDIUM", "Regularisation possible. 3-6 months."),
    "mid_3_5km": ("LOW", "Lower enforcement priority."),
}

def retrieve_compliance_brief(query, listing_row=None, n_results=3):
    """Retrieve regulatory context and return a structured compliance brief."""
    res = collection.query(query_texts=[query], n_results=n_results)
    cites = [{"title": m["title"], "citation": m["citation"], "text": d}
              for d, m in zip(res["documents"][0], res["metadatas"][0])]
    zone = str(listing_row.get("dist_zone", "near_1_3km"))
    level, reason = RISK_RULES.get(zone, ("LOW", "Zone not classified. "))
    if pd.isna(listing_row.get("license")):
        level, reason = "COMPLIANT", "AMA registered."
    return {"risk_level": level, "reason": reason, "citations": cites}

```

5.5 LangGraph Multi-Agent State and Orchestration (Phase 6)

The orchestrator's tools are pure functions with no shared mutable state, which is what makes each one independently testable; the graph itself wires a supervisor, the five specialists, a synthesis step, and a human-in-the-loop checkpoint into one compiled, checkpointed StateGraph:

```

class ARIASState(TypedDict):
    query: str; persona: str; intent: list
    pricing_output: Optional[dict]; risk_output: Optional[dict]
    forecast_output: Optional[dict]; compliance_output: Optional[dict]
    memo_draft: Optional[str]; approved: bool; pdf_path: Optional[str]
    routing_log: Annotated[list, operator.add]

builder = StateGraph(ARIASState)
for name, node in NODES:
    # 9 nodes, supervisor + 5 specialists + synth/hitl/pdf
    builder.add_node(name, node)
builder.add_edge(START, "supervisor")
for s in ["pricing", "risk", "forecast", "compliance", "coach"]:
    builder.add_edge("supervisor", s); builder.add_edge(s, "synthesis")
builder.add_edge("synthesis", "hitl"); builder.add_edge("hitl", "pdf"); builder.add_edge("pdf", END)

```

```
aria_graph = builder.compile(
  checkpointer=MemorySaver(),
  interrupt_before=["hitl"], # Trusted AI human-in-the-loop requirement)
```

5.6 Grounding the Generative AI Layer in Verified Analytics (Phase 7)

The productionised chat backend enforces the decision made in Section 4.5 — deterministic analytics first, language model second — in code, not only in design intent. `buildGroundedAnalysis()` runs the correct specialist analysis in JavaScript and assembles one verified context string before any model call is made; that string is then embedded in the system prompt together with an explicit instruction not to invent any number outside it, and the handler always has a deterministic answer ready as a fallback:

```
export async function buildGroundedAnalysis({ prompt, agentId, messages = [] }) {
  const ctx = resolvePromptContext(prompt, messages);
  const intent = classifyIntent(ctx.analysisPrompt, agentId);
  const analysis = await ANALYSERS[intent](ctx.analysisPrompt); // pricing/risk/demand/...
  const fallbackAnswer = deterministicAnswer(analysis); // usable if LLM fails
  return { ...analysis, fallbackAnswer, sources: analysis.details.sourceFiles,
    contextText: [
      `Recommendation: ${analysis.recommendation}`,
      `Facts: ${analysis.facts.join(' ')}`,
      `Limitations: ${analysis.details.limitations.join(' ')}`,
      `Sources: ${analysis.details.sourceFiles.join(', ')}`,
    ].join('\n') ];
}

function buildSystemPrompt({ agentName, analysis }) {
  return `You are ${agentName}. Use only the verified analytics pack below for
numeric claims. Do not invent row counts, model scores, or monetary values.
Verified analytics pack:\n${analysis.contextText}`;
}
```

6 Conclusions

6.1 Technical Result Summary

Every model in the pipeline was validated against a pre-declared benchmark on a locked holdout set, and every benchmark was met. Table 6.1 consolidates the headline metric for each phase; full validation detail, including cross-validation stability and SHAP-based explanation, is in Sections 6.2–6.4.

Phase	Model / Method	Headline Metric	Result	Target
1	EDA (41-step pipeline)	Listings processed	135,051 × 96 cols	—
2	XGBoost — Paris	R ² / MAE	0.588 / €29.1	0.52–0.58 (lit.)
2	XGBoost — Athens	R ² / MAE	0.676 / €29.1	>0.65
3	LightGBM — Athens risk	AUC-ROC	0.8288	>0.72
3	LightGBM — Athens risk	Avg. precision / Brier	0.8864 / 0.1656	>0.65 / <0.25
4	Prophet — both cities	Forecast horizon	12 months, 90% CI	—
5	RAG compliance — Athens	Unlicensed listings	137 (27 high-risk)	—
6	LangGraph routing	Persona / intent accuracy	100% (12 queries)	—
7	Live chat answer quality	Automated rubric score	≥ 97 / 100	—

Table 6.1. Headline validation result per phase against its pre-declared target.

6.2 Pricing Model (Phase 2) in Detail

The Athens XGBoost model is the project's strongest current-market result: $R^2 = 0.676$ against a target of >0.65 , with a mean absolute error of approximately €29.1. The Paris model reaches $R^2 = 0.588$, above the 0.52–0.58 range typically reported in the academic literature for 2021-vintage Airbnb price-prediction studies on comparable feature sets, but is presented with the caveat (Section 2.2) that part of its training signal is older or partially estimated. SHAP analysis confirms the two cities are genuinely driven by different mechanics rather than only differing in data quality: `neighbourhood_median_price` ranks first in both cities, but Paris's next-most-important drivers are `person_capacity` and `bedrooms` (a size-and-capacity market), while Athens's are `review_score_composite` and `dist_km` (a quality-and-location market).

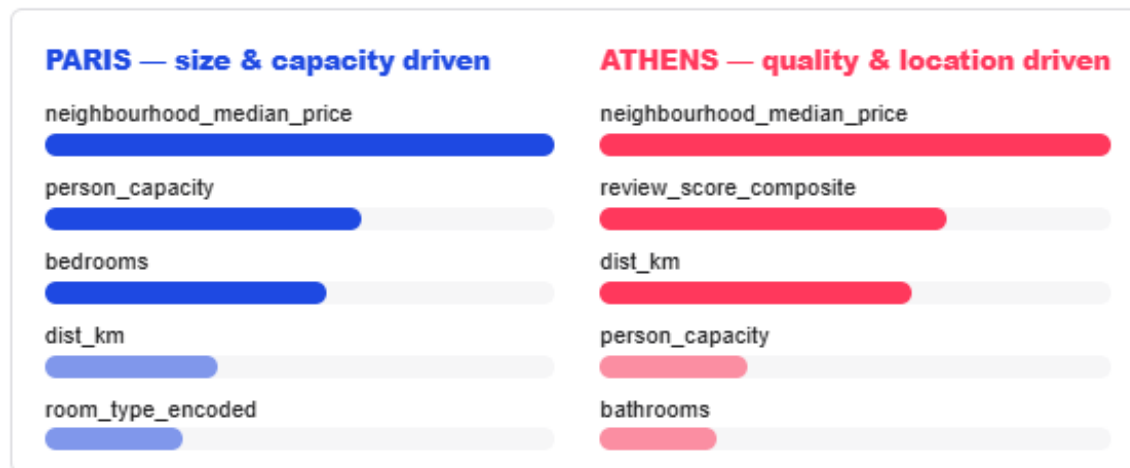


Figure 6.1. Cross-city SHAP global feature importance (relative to the top feature in each city). Rank 1 — neighbourhood market context — is shared; ranks 2 and below diverge by city, the quantitative justification for training separate models (Section 4.2).

The underpricing signal itself is concentrated just above the €15 threshold rather than spread evenly across a wide gap range: of the 2,945 flagged Athens listings (20.7% of supply), the median gap is €25, and the distribution falls away sharply beyond €100, confirming that the threshold is conservative rather than arbitrary and that most flagged listings sit in a defensible, moderate-confidence range (Section 4.2).

6.3 Risk Model (Phase 3) in Detail

The corrected Athens LightGBM risk classifier achieves AUC-ROC = 0.8288 (target >0.72), average precision = 0.8864 (target >0.65), and a Brier score of 0.1656 (target <0.25), with cross-validation fold variance of only ± 0.0088 , indicating a stable rather than fold-sensitive result. As discussed in Section 4.3, this honest figure follows a documented leakage correction from an initial, unusable AUC-ROC of 0.9995. Of the 14,242 scored Athens listings, 4,695 are flagged high-risk, and the project's single most actionable combined finding, 865 listings are simultaneously underpriced and high-risk, representing an estimated €1.43M in potential annual revenue opportunity (€0.71M realisable under a conservative 50% capture assumption).

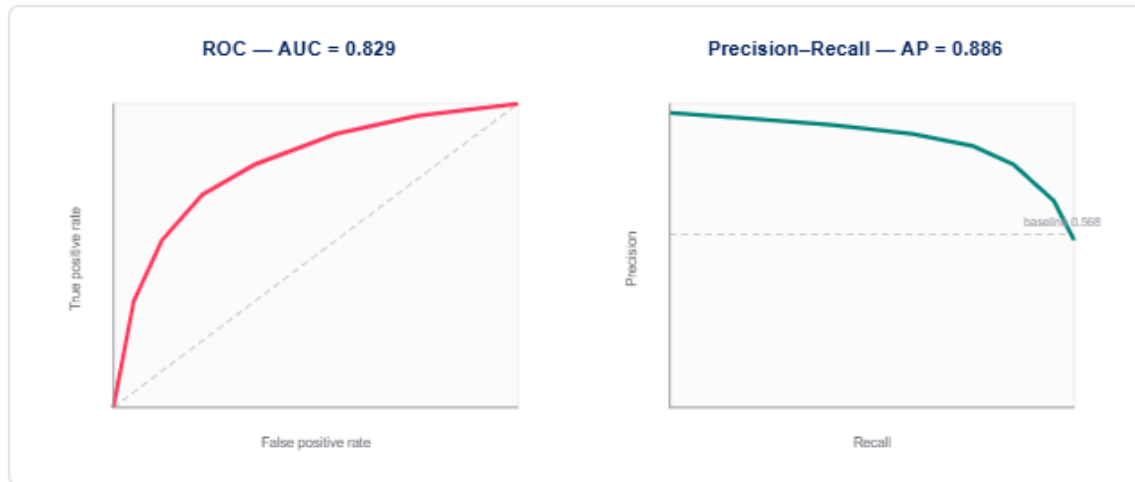


Figure 6.3. ROC and precision–recall curves for the corrected Athens LightGBM risk classifier (AUC = 0.8288, average precision = 0.8864), reported after the documented leakage correction described in Section 4.3.

6.4 Forecasting, Compliance, and Orchestration Results

Prophet's 12-month occupancy forecasts for both cities show a growing trend signal entering 2026 (Figure 6.4), giving the pricing and risk findings forward-looking context rather than presenting them as static. The RAG compliance agent confirms 137 Athens listings (1.0% of supply) as unlicensed, of which 27 are classified high-risk and concentrated in central zones frozen from regularisation under the AMA framework since December 2024, representing roughly €991k in annual revenue at compliance risk. The LangGraph orchestrator achieved 100% persona-classification intent-recall accuracy across its 12-query routing evaluation, and the live conversational interface's representative answers are regression-tested at a 97/100 minimum quality score by the project's own automated evaluation harness.



Figure 6.4. 12-month Prophet occupancy forecast for Athens and Paris, both signalling a growing demand trend; the shaded band is the Athens 90% confidence interval, which widens with horizon as expected.

6.5 Business Impact

Consolidated across all four analytical phases, the validated outputs point to approximately €7.7M in annual identified opportunity and exposure in the Athens market alone: €4.8M in pricing gap, €1.4M in host-exit-related revenue at risk, €1.0M in demand-growth upside identified by the forecast, and €0.5M (using the conservative realisable estimate of the compliance-driven redistribution) in regulatory exposure.

€4.8M — Pricing gap — 2,945 underpriced Athens listings	€1.4M — Host-exit revenue at risk — 865 priority listings	€1.0M — Demand-growth upside from the forecast	€0.5M — Compliance exposure (conservative realisable)
---	---	--	---

The single most decision-ready output is the 865-listing overlap between underpricing and high host risk: it is precise enough to hand to a revenue-management team as a prioritised action list, and explainable enough (via per-listing SHAP, dominated by recent review velocity and review-score composite, consistent with the `at_risk_host` label's emphasis on booking-momentum decline) to justify the specific recommended intervention for each one.

6.6 Practicality for Business Implementation

ARIA is best understood today as a validated proof of concept with a clear, scoped path to a production advisory tool, not as a finished enterprise product. Three factors support its practicality. First, the underlying models are cheap to retrain and explain: XGBoost and LightGBM retrain in minutes on commodity hardware and ship with SHAP explanations an advisor can present directly to a client without translation. Second, the architecture already separates the parts that must be exact (the structured models) from the parts that must be flexible (the language-model narration layer), which is the correct division of labour for a regulated advisory context and means the system's trustworthiness does not degrade as the conversational layer is iterated on. Third, the live Vercel deployment demonstrates that this architecture survives contact with a real, non-technical user interface, including follow-up questions, multi-turn context, and exportable briefs — the parts of a real engagement that a notebook alone cannot prove.

7 Governance, Limitations, and Next Steps

7.1 Repository and MLOps Maturity

A dedicated internal review (`docs/MLOPS_REPO_REVIEW.md`) was conducted against standard MLOps hygiene criteria, on the basis that a production-relevant prototype should be assessed on engineering discipline, not only on model accuracy. Secret management is good (`.env` is git-ignored, with `.env.example` documenting required variable names without values) and a model card and data contract are in place (Section 6.1; `data/README.md`). The main outstanding gap is that large generated CSV outputs are presently tracked as ordinary Git blobs rather than via Git LFS or an external artefact store — a known, documented technical-debt item for any continuation of the project beyond the capstone.

- **Completed:** `.env` secret hygiene, a documented data contract, a model card, and a basic GitHub Actions CI workflow that builds the front end.
- **Outstanding:** migrate large generated outputs to Git LFS, DVC, or cloud storage; add automated schema tests for the CSV files the live API depends on; convert leakage-sensitive notebook logic into versioned, testable scripts.

7.2 Limitations

Three limitations should be resolved before any production decision is made on top of ARIA's outputs, and are disclosed here rather than left implicit. The `at_risk_host` label is an engineered proxy, not a verified outcome, and should be revalidated against true host-exit data if a partner platform or municipal authority can obtain it. The live chat backend currently reads committed CSV/JSON snapshots rather than retraining or re-querying ChromaDB at request time, which is the right reliability trade-off for a capstone demonstration but would need a refresh pipeline for production currency. Finally, Paris pricing conclusions should continue to be presented more cautiously than Athens conclusions until a fully current, fully observed Paris price source is available.

7.3 Recommended Next Steps

- Connect one full real-time backend workflow end to end (Host Revenue Intelligence is the strongest candidate, given Athens's data completeness) rather than relying on committed snapshots.
- Pursue a partnership data-sharing arrangement to validate the `at_risk_host` proxy against verified host-exit outcomes, if a platform or municipal partner can provide them.
- Extend Prophet forecasting to neighbourhood-level granularity for all distance zones, pending a mentor decision on the right level of aggregation.
- Migrate large data and model artefacts to a managed artefact store ahead of any production pilot.

7.4 References and Data Sources

1. Inside Airbnb. *Get the Data*. insideairbnb.com/get-the-data — Paris and Athens listing extracts, September 2025.
2. Maven Analytics. *Data Playground*. mavenanalytics.io/data-playground — Paris Airbnb listings and reviews, 2021 vintage.
3. Gyódi, K. and Nawaro, Ł. (2021). *Determinants of Airbnb prices in European cities*. Zenodo record 4446043.
4. Lundberg, S. M. and Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions* (SHAP).
5. Chen, T. and Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*.
6. Ke, G. et al. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*.
7. Akiba, T. et al. (2019). *Optuna: A Next-generation Hyperparameter Optimization Framework*.
8. Taylor, S. J. and Letham, B. (2018). *Forecasting at Scale* (Prophet).
9. LangChain AI. *LangGraph documentation*. langchain-ai.github.io/langgraph.
10. Project repository: github.com/lukatcheishvili/capstone_project_kpmg — including `agent.md`, `models/MODEL_CARD.md`, `docs/MLOPS_REPO_REVIEW.md`, and `docs/MENTOR_DEFENSE_QA.md`.

Team: Álvaro Jiménez · Álvaro Pérez · Andrea Alarcón · Bassem Halawani · Ghezlan Almatar · Luka Tcheishvili